老师反馈技术补充

2026-10-25 更新。范围为规划书 Week 1–4；反馈口述中的周次与源码归属分别记录，不搬动已有模块以匹配口述编号。

UART 波特率计算与寄存器验证

本项补齐 Week 3–4 UART0 的计算依据、取整过程、寄存器读回及仿真边界。专项验证对两个阶段的生产固件分别干净构建并运行，两项均通过。Week4 使用全 FF 镜像使 Bootloader 停在拒绝分支，读回的是 Bootloader 初始化后的寄存器，不是应用重新初始化后的状态。

公式与取整

本工程外设为 CMSDK APB UART。设 UART 输入时钟为 fPCLK、目标波特率为 Btarget、整数分频为 D，则 Dideal = fPCLK / Btarget；分频后的理论波特率 Bactual = fPCLK / D。此 BAUDDIV 直接记录时钟分频值，无需在分母额外乘 16。依据见 Arm CMSDK 手册 4.3 节、表 4-7 及 4-10 页的分频示例。Arm CMSDK 技术手册 DDI0479D

输入 fPCLK = 25000000 Hz，Btarget = 115200 baud：

Dideal = 25000000 / 115200 = 217.0138888889
整数商 q = 217
余数 r = 25000000 - 217 * 115200 = 1600
四舍五入阈值 = ceil(115200 / 2) = 57600
1600 < 57600，因此不进位，D = 217 = 0x000000D9

实际代码在 src/week3/uart.c 的 uart_init 中先做整除，再判断 r >= baud - baud/2 才进位；采用商和余数避免 clock_hz + baud/2 的加法溢出。两个输入须非零，取整后 D 须落在[16, 0xFFFFF]；不合格时返回-1，在修改寄存器之前退出。Week4 复用同一驱动。此处记录源码规则，本次专项只新增默认配置的读回验证，不声称重跑了所有非法参数测试。

实际波特率及量化误差

Bactual = 25000000 / 217 = 115207.3732718894 baud
差值 = Bactual - Btarget = +7.3732718894 baud
相对误差 = (Bactual - Btarget) / Btarget
= 0.00006400409626
百分比误差 = 相对误差 * 100% = +0.006400409626%
ppm 误差 = 相对误差 * 1000000 = +64.00409626 ppm
理论位宽 = D / fPCLK = 8.68 us
8N1 连续发送一帧为 10 位，理论帧时间 = 86.8 us（不含帧间空闲）

正号表示分频结果略快于目标。误差只含整数分频量化误差，未计入晶振偏差、PLL/时钟树误差、抖动和对端时钟偏差，不能据此直接判断真机链路的总体容差。

寄存器读回与证据

复验命令：在 codes/ 执行 python3 scripts/verify-uart-baud.py。脚本在临时目录构建两个 Bootloader，用 QMP 的 info qtree 定位 MMIO 基址 0x40004000 对应的 UART0，读取其 pclk-frq，随后经 GDB 读回 32 位寄存器并断言。证据根目录 BAUDLOG = logs/uart-baud-20261024-235338（相对 codes/）。该目录也保存构建输出、QEMU 版本、GDB 命令文件和 ELF SHA256；本轮 QEMU 为 11.1.2。

固定编号	操作与具体输入	预期	实际与判定
BAUD-W3	干净构建 Week3；默认空白存储；启动至 UART 提示符；读 qtree 和寄存器	UART0 pclk-frq=25000000；BAUDDIV=217；CTRL=0x0B	25000000；0x000000D9 (217)；0x0000000B；PASS
BAUD-W4	干净构建 Week4；全 FF 镜像；停在 REJECT 后读 qtree 和寄存器	同上，读取 Bootloader 状态	25000000；0x000000D9 (217)；0x0000000B；PASS

BAUD-W3 证据：BAUDLOG/week3-qtree.txt、BAUDLOG/week3-registers.gdb、BAUDLOG/week3-registers.txt、BAUDLOG/week3/uart.log。BAUD-W4 对应 week4 同名文件；汇总为 BAUDLOG/summary.json。两个 registers.txt 均包含 UART_BAUD_READBACK_PASS。

UART0 基址为 0x40004000；BAUDDIV 偏移 0x10，读回地址为 0x40004010。CTRL 偏移 0x08，读回地址 0x40004008；0x0B 表示 bit0 TX 使能、bit1 RX 使能、bit3 RX 中断使能，bit6 高速测试模式未开启。GDB 可用 x/wx 0x40004010 和 x/wx 0x40004008 人工复核。

仿真验证边界

本轮证明模型 UART0 的时钟属性、固件写入的分频和使能寄存器符合设计，且启动字符可通过模型后端接收。QEMU 11.1.2 的 cmsdk-apb-uart.c 通过整数除法将 pclk_frq/bauddiv 传给字符后端的串口参数接口，数值为 115207；它与上述精确分频推导值的表示精度不同。QEMU 11.1.2 UART 模型源码

本工程使用 Unix socket 字符后端，接收的是字节流，没有测量物理 TX/RX 波形。因此终端字符正确、PONG 应答成功、宿主日志时间间隔都不能作为 115200 物理线速、8.68 us 位宽、对端容差或实物时钟准确性的实测证据。真机验收仍需确认实际 UART 输入时钟，并用示波器或逻辑分析仪测量重复 0x55 等数据的 TX 位宽，按 Bmeasured = 1/Tbit 计算实测波特率，再与目标及对端容差比较。此项尚未开展。

PRIMASK 保存与恢复接口契约

接口位于 src/week2/irq.h，由 src/week3/uart.c 的初始化和队列消费复用。此公共基础接口已在前轮整理到前一阶段，本次仅补注释和可追溯测试，不移动文件，不改变驱动执行逻辑。第二阶段轮询 UART 没有接收队列临界区，不为形式统一增加调用。

调用约束与效果

uint32_t irq_save(void)：读取进入时 PRIMASK，返回 0 或 1，随后通过 CPSID i 设置 PRIMASK=1。调用返回后才进入受保护区；读取和屏蔽是两条指令，不应把读取瞬间视为已经屏蔽。

void irq_restore(uint32_t saved)：通过 MSR PRIMASK 写回对应 irq_save 返回的原值。参数必须是这一层保存的值；本接口不做参数校验。正常、错误和提前返回路径都须配对恢复。嵌套必须后进先出，内层恢复内层值、外层恢复外层值；不得用固定 0 替代保存值，否则调用者原本屏蔽中断时会被错误解除屏蔽。

适用范围为单核 Cortex-M 特权执行环境。PRIMASK=1 阻止可配置优先级异常激活，不屏蔽 NMI 和 HardFault；恢复 0 仅解除 PRIMASK 这一层屏蔽，不保证 NVIC、BASEPRI 及 FAULTMASK 等其他条件允许中断立即执行。接口不改这些寄存器，也不清除 pending 状态。语义依据：Arm CMSIS Core 寄存器说明。

内联汇编的 memory clobber 约束编译器对内存访问的重排，不等同于 DMB 或 DSB 硬件屏障。该接口不提供多核互斥，不替代 Bootloader 跳转时的完整处理器状态交接；设备操作需要的硬件屏障由调用处按需求安排。临界区应保持短小，当前 UART 消费端只访问队列和索引。

```
uint32_t outer = irq_save();
/* 外层临界区 */
uint32_t inner = irq_save();
/* 内层临界区 */
irq_restore(inner); /* 仍为 1 */
irq_restore(outer); /* 回到调用者的 0 或 1 */
```

专项验证方法

在 codes/ 执行 python3 scripts/verify-irq-contract.py。脚本临时交叉编译实际 src/week3/uart.c 与 tests/test_irq.c，在 QEMU AN500 Cortex-M7 中执行 MRS、CPSID、MSR 指令。初始状态分别用独立 CPSIE i 及 CPSID i 建立，不用被测 irq_restore 设置前置条件。每组 11 个断言，每个断言打印实际值、预期值和 OK/FAIL；两组全部通过，QEMU 退出 0。

证据根目录 IRQLOG = logs/irq-contract-20261024-235737（相对 codes/）：build.txt 记录编译命令及退出码，qemu.txt 记录完整逐项结果和退出码，summary.json 记录两组判定与 ELF SHA256，qemu-version.txt 记录版本。测试用 semihosting 输出结果并退出，仅用于测试固件，不引入生产固件。

固定编号	输入与操作步骤	预期结果	实际结果与证据
IRQ-INIT-0	CPSIE i 建立 0；外层 save；内层 save；内层 restore；外层 restore；实际 uart_init；空队列 uart_getc	外层保存 0；内层保存 1；内层恢复 1；外层恢复 0；UART 调用后保持 0	11 个断言全部 OK；PASS；IRQLOG/qemu.txt 中 GROUP initial=0
IRQ-INIT-1	CPSID i 建立 1；依次执行相同操作	外层保存 1；内层保存 1；两层恢复都为 1；UART 调用后保持 1	11 个断言全部 OK；PASS；IRQLOG/qemu.txt 中 GROUP initial=1

下表每一行对应日志中的同名断言。两列使用“预期/实际”格式，所有行均为 PASS；联合固定编号由组编号与断言名构成，例如 IRQ-INIT-0/OUTER_RESTORE。

断言名	检查点	初值 0 预期/实际	初值 1 预期/实际
ENTRY	独立设置后读 PRIMASK	0/0	1/1
SAVE_RETURN	外层 irq_save 返回值	0/0	1/1
SAVE_MASK	外层 save 后读 PRIMASK	1/1	1/1
NEST_RETURN	内层 irq_save 返回值	1/1	1/1
NEST_MASK	内层 save 后读 PRIMASK	1/1	1/1
INNER_RESTORE	恢复内层后读 PRIMASK	1/1	1/1
OUTER_RESTORE	恢复外层后读 PRIMASK	0/0	1/1
UART_INIT_RETURN	uart_init(25000000,115200)返回值	0/0	0/0
UART_INIT_MASK	初始化调用后读 PRIMASK	0/0	1/1
UART_GETC_RETURN	空接收队列 uart_getc 返回值	0/0	0/0
UART_GETC_MASK	队列消费调用后读 PRIMASK	0/0	1/1

本轮验证范围为默认配置、空队列路径、两层嵌套和特权模式下的掩码读回。不将其扩展为非特权调用有效性、NMI/HardFault 注入、所有异常出口或真机中断延迟的测量结论。既有 FB-IRQ-02 仍保留历史证据；当前 tests/test_irq.c 仍输出两条 PASS 汇总，兼容旧补测脚本的检查条件。

TX 持续阻塞的有限轮询与错误传播

2026-10-25 补测直接覆盖持续 TX_FULL，7 组专项检查全部通过：两版驱动各 10 个断言、3 个生产目标隔离检查，以及 Week3 输出、Week4 输出、Week4 跳转前 flush 和 Week4 应用输出四条错误路径。每组都有固定编号、输入、预期和实际结果及日志。本轮重点为错误传播及有限退出，不把宿主看门狗终止当作驱动超时成功。

注入方式与复验

在 codes/ 执行 python3 scripts/verify-tx-full.py。测试在临时工程副本中建立独立测试目标，启用 UART_TX_FAULT_TEST，使用 tests/uart_tx_test.h 及 tx_fault_state.c 提供的易失状态、DATA 替身和状态读取计数器。src/week2/uart.c 与 src/week3/uart.c 仍执行原有 uart_flush、uart_putc 和 uart_write 函数；没有另写一套轮询算法。注入使每次 TX 状态读取都返回 1，持续到函数自行返回。

两版驱动均交叉编译到 Cortex-M7，在 QEMU 上执行。第三阶段 Bootloader 及第四阶段 Bootloader 复用 week3 驱动，第四阶段应用使用 week2 驱动。完整上层测试复用实际 main 和错误处理代码，通过 GDB 在

idle 函数处读取状态；初始测试 DATA 为 0x55，禁用生产 UART 输出对 DATA 替身的物理发送。不能仅以 UART 没有文本作为成功证据，须同时满足返回值、读取次数和状态断言。

正常构建不定义 UART_TX_FAULT_TEST，直接访问原 MMIO 寄存器。正常 bootloader-week3.elf、bootloader-week4.elf、application.elf 均重新构建，并通过符号表确认不含 uart_tx_test_*。本次生产路径只增加编译期选择，不改变轮询上限和错误语义。

证据根目录 TXLOG = logs/tx-full-20261025-015904（相对 codes/）。summary.json 汇总 7 组结果及 ELF SHA256；每个.txt 日志保存实际命令、完整输出及退出码；各错误路径同时保存.gdb 脚本和-uart.log。

驱动返回值与计数

两组均先初始化 UART 至 TX 使能，设置 TX_FULL=1、读取计数=0、DATA=0x55；依次调用 flush、putc('A')、write("ABC")，随后令 TX_FULL=0，重新发送"A"。下表每项分别在 TX-WEEK2-DRIVER 和 TX-WEEK3-DRIVER 下执行，联合编号为组编号/断言名，20 个断言全部 OK。证据分别为 TXLOG/week2-driver.txt、TXLOG/week3-driver.txt。

断言名	具体输入或操作	预期结果	两版实际结果
INIT	uart_init(25000000,115200)	返回 0	0；OK
FULL_FLUSH_RETURN	持续 TX_FULL=1；uart_flush()	-1	-1；OK
FULL_FLUSH_READS	上一步返回后读计数	1000000	1000000；OK
FULL_PUTC_RETURN	uart_putc('A')	-1	-1；OK
FULL_PUTC_TOTAL_READS	flush 和 putc 累计状态读取	2000000	2000000；OK
FULL_WRITE_RETURN	uart_write("ABC")	首字符发送失败即返回-1	-1；OK
FULL_WRITE_TOTAL_READS	三次调用后累计状态读取	3000000，未继续发送 B/C	3000000；OK
FULL_DATA_UNCHANGED	读取测试 DATA	仍为 0x55，无失败写入	0x55；OK
RECOVER_WRITE_RETURN	TX_FULL=0，计数清零；write("A")	0	0；OK
RECOVER_DATA_READS	恢复后读 DATA 和计数	DATA=0x41，读取 1 次	0x41、1；OK

两组 QEMU 测试均由固件断言完成后主动退出 0，包含 TX_FULL_CONTRACT_PASS；脚本 10 秒外层限制仅防止测试挂死，本轮未触发。

上层失败状态与成功标记

固定编号	操作步骤与具体输入	预期结果	实际结果
TX-PRODUCTION	重新构建 3 个正常目标；nm 检查符号	无注入变量	三个 ELF 均不含 uart_tx_test_*；PASS
TX-W3-PRINT	持续 FULL=1、DATA=0x55；运行实际 Week3 main；断在 fault_idle	print 收到 write 失败；boot_status=3；有限退出	boot_status=0x3；读取 1000000 次；DATA=0x55；PASS
TX-W4-PRINT	有效镜像，持续 FULL=1；运行实际 Week4 main；断在 boot_idle	初次输出失败；boot_status=0x101；不跳转应用	0x101；读取 1000000 次；DATA=0x55；PASS
TX-W4-FLUSH	有效镜像；FULL=0 完成校验和输出；在 main 的最终 flush 前改为 FULL=1	最终 flush 失败，boot_status=0x101；停在 boot_idle	0x101；该次 flush 新增读取 1000000 次；PASS
TX-W4-APP	正常 Bootloader 加载有效的注入应用；应用 FULL=1；断在 app_idle	交接正常；应用 print 失败；app_status=4；不进入运行成功状态	handoff_ok=1；app_status=4；读取 1000000 次；DATA=0x55；PASS

TX-PRODUCTION 证据为 TXLOG/bootloader-week3-symbols.txt、bootloader-week4-symbols.txt、application-symbols.txt。其他 4 组各对应 TXLOG/编号.txt、编号.gdb、编号-uart.log。例如 TX-W4-FLUSH 对应 TXLOG/TX-W4-FLUSH.txt。

四条错误路径 GDB 断言均退出 0，均没有 PONG 或 APP: RUNNING。Week4 应用用例允许 Bootloader 先输出自身的 CRC/LOAD 成功信息；最终是否运行成功须同时检查 app_status 和 APP: RUNNING，不能把之前的中间成功行当作应用成功。GDB 取证后由测试脚本终止并回收 QEMU，不能将这个宿主清理退出码当作固件成功退出。

轮询预算的边界

一百万次限制是每次 uart_flush 最多读取状态一百万次的预算，不是 100 万毫秒，也不是固定的毫秒超时。执行时长受 CPU 时钟、编译优化、MMIO 访问开销、中断抢占及 QEMU 调度影响。测试计数器本身也增加开销，因此不把本轮宿主耗时换算为真机超时。

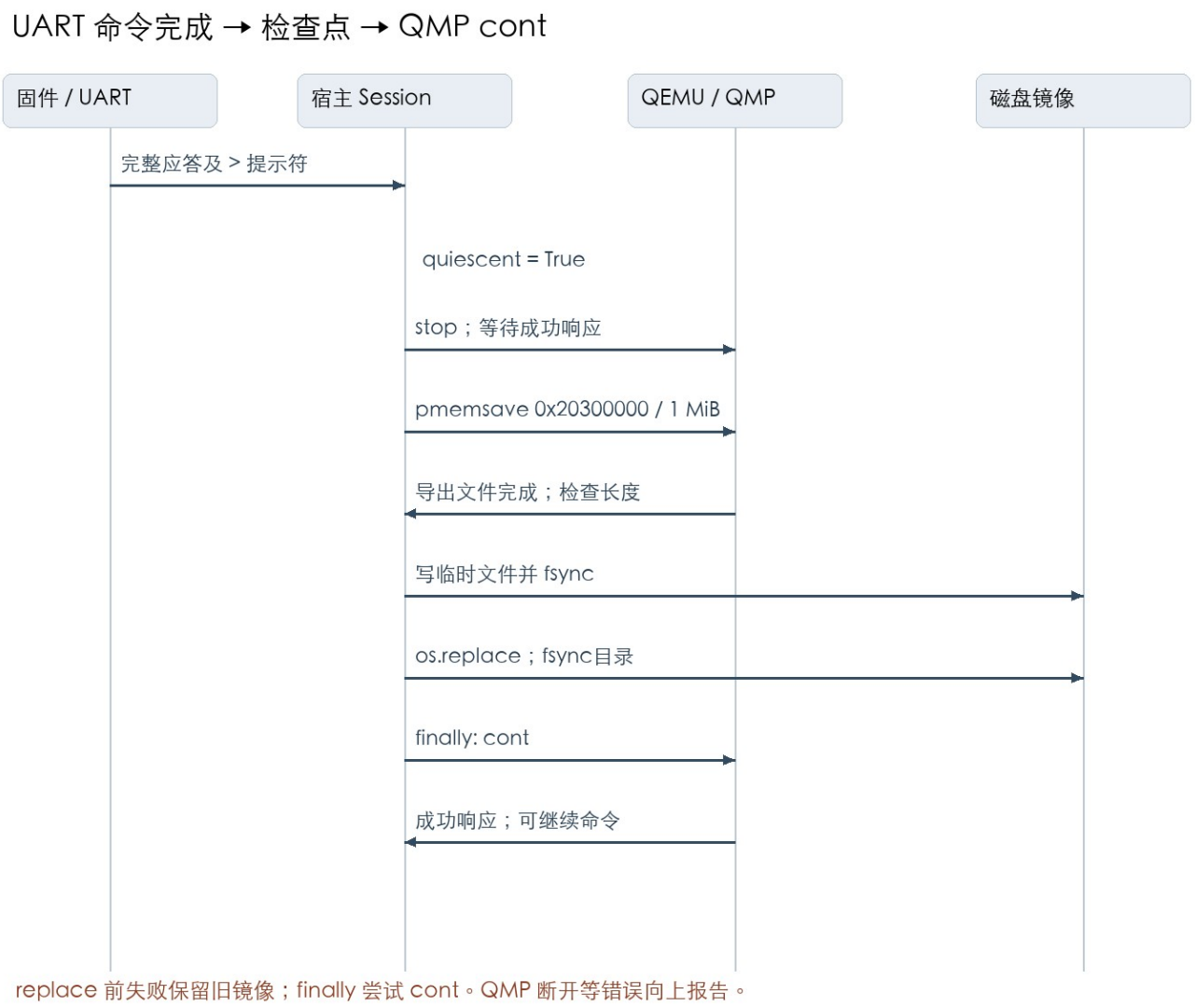
uart_putc 先 flush，失败时不写 DATA；uart_write 对非空字符串逐字符调用 putc，遇到首个失败立即返回-1。本次持续阻塞在首字符即发生；空字符串没有待发送字节，不要求因 FULL 而失败。发送多字节的总预算与字符数、阻塞位置有关，调用者另行调用 flush 也有独立预算。若产品要求准确毫秒超时，应使用明确时基和截止时间，并对抢占、时钟变化和计数回绕另行设计验证。

故障注入替代的是 UART 状态和 DATA 观测点，验证了 Cortex-M7 上驱动控制流及实际调用者的错误状态。它没有制造 QEMU 字符后端的物理持续拥塞，也未验证真机串口线上的阻塞时长。UART 失效时不依赖再发送一条错误文本，使用 GDB 读取 boot_status/app_status 作为证据。

正常路径回归

未启用注入的第三阶段 11 项、第四阶段 27 项回归均通过。新证据为 logs/week3-20261025-015915-135199 和 logs/week4-20261025-015917-068252。既有逐项报告保留原始证据路径；这两份目录是本次新增的生产分支回归证据，不替换先前日志。

UART 命令完成到 QMP cont



UART 检查点时序

固件处理命令，输出完整响应及提示符“> ”。宿主 response 收到提示符才将 quiescent 置真；checkpoint 要求该状态，然后发送 stop 并等待成功响应，发送 pmemsave 导出 0x20300000 起 1MiB。宿主检查长度，写同目录临时文件、fsync 文件、os.replace 原子替换、fsync 目录。finally 发送 cont。保存中断或 replace 失败时

会尝试恢复 QEMU；replace 前失败保持旧镜像。若目录 fsync 在替换后失败，则新文件可能已经可见，不承诺回退旧文件；若 QMP 断开则 cont 可能失败，向调用者报告错误。

本实现只有一个同步命令发送者；提示符证明本轮命令处理完成，不等同于任意并发设备均静止。W3-06/07/08/09 覆盖成功、replace 失败恢复及重启，qmp.json 同时保留请求和响应。

向量合法范围与边界覆盖

本工程固定 AN500、48 项向量和 192 字节向量表。范围规则来自 src/week4/image.c 的 vectors_ok，image_validate 与 boot_verify_loaded 共同调用；宿主打包器 scripts/image_format.py 实施相同的输入规则。下表是本工程镜像契约，不将其扩展为所有 Cortex-M 芯片的统一地址规则。

检查对象	精确规则	边界含义	验证与证据
向量表位置	payload 复制到 0x00010000；向量表 [0x10000,0x100C0)，共 48 项	load 必须为固定地址；表本身按 256 字节对齐	W4-BUILD、W4-HANDOFF；链接与 GDB 证据
初始 MSP vector[0]	0x20010000 < MSP <= 0x20200000；MSP 模 8=0	下界不接受；上界接受；保留 Bootloader RAM	VEC-001..015；MSP-AT-BASE
Reset vector[1]	entry 位 0 必须 1；addr=entry 按位与 0xFFFFFFFE	Thumb 位与地址分别检查，不按 4 字节对齐处理	Reset 索引 7 个边界；RESET-BELOW
所有已实现异常及 IRQ 入口	0x100C0 <= addr < 0x10000+payload_size，且 entry 位 0=1	代码下界接受；上界不接受；零入口拒绝	42 个必填索引各 7 项；VEC-MATRIX
核心保留项	索引 7、8、9、10、13 必须为 0	每一项单独验证 0 可接受、非 0 拒绝	5 项各 2 个断言
外部 IRQ	索引 16..47 对应 IRQ0..31，每项必须为非零合法 Thumb 入口	未使用 IRQ 也要配置合法默认处理函数	全部 32 项都测，不仅测 IRQ0
复制后的复核	先 CRC，再用同一规则检查 MSP 及全部入口	先修正测试 CRC 才能直接触达向量拒绝分支	LOAD-VECTOR-MSP、LOAD-VECTOR-IRQ31

新证据 VLOG = logs/vector-bounds-20261025-021521（相对 codes/）。python3 scripts/verify-vector-bounds.py 在干净构建中执行 321 个宿主断言（ASan/UBSan），再做 4 个 CPU 镜像拒绝、2 个复制后向量复核及 2 个状态路径验证。完整 321 项的输入、预期、实际、文件行号见 codes/docs/vector-boundary-cases.json；原始输出为 VLOG/matrix.txt。

覆盖类别	输入及预期	本轮结果
MSP 上下界	0、base-8、base、base+8、top-8、top、top+8、0xFFFFFFF8	只有 base+8、top-8、top 接受；8 项 PASS
MSP 对齐	base+8 再分别加 1..7	7 种余数全部返回 7；PASS

覆盖类别	输入及预期	本轮结果
每个必填入口	最低合法 Thumb 地址、低于代码区、最高内部地址、等于末端、超过末端、清 Thumb 位、零	42×7=294 项 PASS
每个保留项	零与 0x100C1	5×2=10 项 PASS
表长边界	payload=191、192	191 返回长度错误 4；192 无代码可容纳合法入口，返回向量错误 7；2 项 PASS
CPU 拒绝	MSP 恰为 RAM 下界、保留项 13 非零、IRQ31 恰到末端、Reset 低于代码区	各输出 REJECT: 0x00000007；4 项 PASS
加载后复核	运行区 MSP 改为下界、IRQ31 入口改零；更新 boot_image.crc 以越过 CRC 检查	两项均在 boot_verify_loaded 返回 7；PASS

对于基准 972 字节 payload，代码起始地址范围为[0x100C0,0x103CC)；最低合法 entry 为 0x100C1，最高合法 entry 为 0x103CB，entry=0x103CD 解码为末端，拒绝。接受边界值的宿主用例只验证地址判定，没有把修改成任意内部地址的镜像启动执行。当前规则只检查入口起始地址、Thumb 位及范围，不证明该地址上的完整指令或函数语义有效；实际应用运行证据仍来自未修改的合法应用。最低可执行镜像还须包含代码，不能把 192 字节长度字段检查通过解释为整个镜像通过。

完整状态转移图



Bootloader 异常：记录 exception_number，boot_status = 0x100 → boot_idle；通常没有 REJECT 行。
boot_status = 0 在跳转前再次写入，单看 0 不能区分“尚未初始化状态”和“已交接”。
UART 在 REJECT 输出途中失败：可能仅输出部分文本，最终 boot_status 被改写为 0x101。

完整启动状态及失败去向

图中所有 Bootloader 失败最终都进入 boot_idle 循环，应用自己的成功或失败最终进入 app_idle。当前没有备份槽切换、重试下载或自动恢复分支。完整可编辑图源为 codes/docs/images/boot-state-complete.mmd，渲染脚本为 codes/scripts/render-boot-state.py。图中“通过”表示继续下一步，不表示最终应用运行成功。

boot_status 不是进度枚举：它在 BSS 初始化时为 0，成功交接前也会被写为 0；中间校验阶段没有逐状态写入。判断应用成功须结合 APP: RUNNING、app_status=1 及交接/CRT/SVC 检查。应用出错时 boot_status 可能仍为 0，应读取 app_status 和 app_exception。

REJECT 输出与状态字典

src/week4/main.c 的 reject(result)先把 result 写入 boot_status，再依次输出前缀、十六进制码、CRLF 和提示符，最后进入 boot_idle。精确格式如下，其中 h 为小写十六进制数字；引号表示字符串边界，不是发送字节：

"REJECT: 0xhhhhhhhh\r\n> "

前缀中的空格、0x、恰好 8 位十六进制和末尾提示符后的空格都是输出的一部分。\r\n 表示 CR/LF 两个字节，不是发送反斜线字符；提示符之后没有额外换行。唯一字段是无符号 32 位错误码，来源是 image_result，不含地址、时间、序号，也不是宿主进程退出码。

boot_status 值	名称与触发条件	REJECT 行为	最终停留及证据
0	BSS 初值，或完成最终 flush 后准备交接	不输出 REJECT；单值 0 不证明应用成功	初期位置不定；交接后应用 app_idle；W4-HANDOFF
1	IMAGE_IO：设备未绑定、读取失败等	REJECT: 0x00000001	boot_idle；VLOG/STATUS-IO.txt 及-uart.log
2	IMAGE_FORMAT_ERROR：magic、format、header_size 不符	REJECT: 0x00000002	boot_idle；IMG-02..05
3	IMAGE_HEADER_CRC：前 28 字节 CRC 不匹配	REJECT: 0x00000003	boot_idle；IMG-06
4	IMAGE_LENGTH：长度小于表、超过槽/执行区，或 load/verify 重复边界无效	REJECT: 0x00000004	boot_idle；IMG-07/08；load 参数分支按源码映射
5	IMAGE_ADDRESS：load 不等于 0x10000	REJECT: 0x00000005	boot_idle；IMG-09
6	IMAGE_PAYLOAD_CRC：存储 payload CRC 不匹配	REJECT: 0x00000006	boot_idle；IMG-10
7	IMAGE_VECTOR：存储或运行区向量非法	REJECT: 0x00000007	boot_idle；向量矩阵及 LOAD-VECTOR 两项
8	IMAGE_COPY_CRC：运行区 CRC 与已校验 info.crc 不符	REJECT: 0x00000008	boot_idle；W4-COPY

boot_status 值	名称与触发条件	REJECT 行为	最终停留及证据
0x100	CPU 异常：记录 exception_number=IPSR，再写状态	异常路径不调用 reject，无 REJECT 保证	boot_idle；VLOG/STATUS-HARDFAULT.txt 读回 exception_number=3
0x101	UART 初始化、write 或最终 flush 失败	UART 不可用，不保证错误文本或完整 REJECT	boot_idle；TX-W4-PRINT、TX-W4-FLUSH

VLOG = logs/vector-bounds-20261025-021521（相对 codes/）；IMG 编号对应第四阶段输入表的宿主和 UART 日志，TX 编号对应 TX 专项报告。状态 1 由 GDB 在 image_validate 入口将已绑定设备的 memory 指针置零触发实际 flash_read 失败；0x100 由无效取指触发真实 HardFault 验证。它们分别是数据读取错误注入和 CPU 异常测试，不是篡改 boot_status 伪造结果。各 load 参数失败的所有子条件尚未逐一做 GDB 注入，应以表中具体验证范围为准。

应用使用独立状态：app_status=0 为初值，1 为 CRT/交接/SVC 通过后的运行状态，2 为 CRT、交接或 SVC 检查失败，3 为应用异常（记录 app_exception），4 为 UART 失败。最终均在 app_idle 循环。app_status=4 已由 TX-W4-APP 验证；不能将应用状态值套用为 Bootloader 的 image_result。

格式错误示例（只展示拒绝行及提示符）：

"REJECT: 0x00000002\r\n> "

复制损坏示例的完整 UART 为：

BOOT: stage4 image validation\r\n
IMAGE: CRC32 PASS\r\n
"REJECT: 0x00000008\r\n> "

UART 在 reject 输出途中失败时，可能留下不完整前缀，boot_status 最终被 print 改写为 0x101；这时以 GDB 状态取证，不要求再靠同一 UART 输出错误行。IRQ 唤醒后 idle 函数仍循环，不自动恢复启动流程。

独立 manifest schema 与示例

交付 manifest 是包根目录的 delivery-manifest.json，用于记录列出的交付文件路径、字节数和 SHA256，帮助老师检查复制完整性及复核材料。它不进入 Bootloader 解析流程；固件仍使用 32 字节 M7IM 二进制头。manifest 不是签名或老师批准证明，approval_status 固定为 pending-teacher-review。

独立 schema 路径为 codes/schemas/delivery-manifest.schema.json，采用 JSON Schema Draft 2020-12；实现为 codes/scripts/delivery_manifest.py。校验器实现本 schema 使用的 type、required、additionalProperties、const、enum、pattern、minItems、minimum 及 date-time 检查，不作为任意 schema 的通用执行器。

字段	约束	本轮检验
根对象	6 字段全部必填；不允许额外字段	各字段缺失及 UNKNOWN 均拒绝
schema_version	常量整数 1，布尔 true 不作 1 接受	VERSION、BOOLEAN-VERSION
plan_group	week1-2、week3、week4；week3-4 仅兼容历史合并包	GROUP 拒绝 week5
generated_at	ISO 日期时间，必须+08:00；秒必填，可有小数；日期必须真实有效	DATE、DATE-TEXT、ZONE
image_format	常量 M7IM-v1	FORMAT
approval_status	常量 pending-teacher-review	APPROVAL 拒绝 approved
files	非空数组；每项只允许 path、size、sha256，均必填	FILES-TYPE、EMPTY、FILE-MISSING 等
files[].path	相对 POSIX 路径；无绝对路径、盘符、反斜线、控制字符、.或..分段、双斜线；规范化后不变	TRAVERSAL、ABSOLUTE、DOT 等
files[].size	非负整数，拒绝布尔和小数	SIZE、BOOL-SIZE、FLOAT-SIZE
files[].sha256	64 个小写十六进制字符	HASH、HASH-UPPER

规范化路径、重复路径拒绝、真实文件存在性、长度/哈希、符号链接越界属于校验器的附加语义检查。基本字段形状由 schema 约束；schema 本身不读取文件。生成器扫描目录中的文件，排除 delivery-manifest.json 自身以避免自引用；校验器核对清单列出的文件，不声称对未列出的额外文件做完整性认证。

有效示例完整 JSON 如下。配套 README.txt 字节存放在 codes/schemas/examples/，其长度和 SHA256 与清单一致。


```
{
"schema_version": 1,
"plan_group": "week3-4",
"generated_at": "2026-10-25T02:16:04.132229+08:00",
"files": [
{
"path": "README.txt",
"size": 25,
"sha256": "798fea15a540e232f4ae06c1888d0467da504195abb547083ebfec97ceb955fb"
}
],
"image_format": "M7IM-v1",
"approval_status": "pending-teacher-review"
}
```


无效示例：缺少必填 files 字段，应抛出 ValueError，不能作为空清单接受。

{
"schema_version": 1,
"plan_group": "week3-4",
"generated_at": "2026-10-25T02:16:04.132229+08:00",
"image_format": "M7IM-v1",
"approval_status": "pending-teacher-review"
}

完整的另外两份无效 JSON 为 schemas/examples/invalid-traversal.json 和 invalid-hash.json，分别把 path 设为../escape 和 sha256 设为 xyz。有效与三个无效示例均在本轮脚本中生成并验证；校验成功 0 退出，字段/文件失败抛异常，命令非零退出。

复验：在 codes/执行 python3 scripts/verify-manifest.py。本轮 41 项全部通过，证据 MLOG = logs/manifest-20261025-021604（相对 codes/），每项完整输入 JSON 和预期/实际异常见 summary.json 及对应 JSON 文件，包括有效实例、6 个必填字段、各标量/数组类型、日期、大小、哈希、路径、重复、缺失文件与符号链接越界。

生成正式交付清单：python3 scripts/delivery_manifest.py 包目录 --group week4；再次核验：python3 scripts/delivery_manifest.py 包目录。打包后还校验 ZIP 完整性。清单能检测非故意复制错误，但 SHA256 清单可被一同修改，不具备签名认证能力。

修改记录与版本追踪

逐项记录位于 planning/teacher-feedback-2026-10-24.md：每条均列反馈编号、修改位置、验证用例、证据路径、修订版本和老师确认状态。版本清单位于 planning/feedback-version.json，记录 Git 基线、待提交状态及本轮源文件内容指纹。老师确认日期和结论由实际反馈填写，当前统一待确认，不用测试通过替代老师批准。

2026-10-25 归档补充：Week3 与 Week4 周报现分别位于 reports/week3 与 reports/week4。交付清单 plan_group 新增 week3、week4，保留 week3-4 兼容历史快照；此枚举扩展不改变原有校验约束。原跨周测试证据及实例保留执行时版本。